

Spam Email Detection Using Naive Bayes Classification

Name	:	Sadman Ishtiak
Roll	:	22102009
Reg	:	10815
Session	:	2021-22

Folder Cleanup and Preparation

First we delete the thngs that we have in the memory currently. The program is tested and written on debian based linux.

```
!rm *.csv.*
!rm *.csv

rm: cannot remove '*.csv.*': No such file or directory
```

Data Collection

Then we downdload the data that is stored in the internet publicly.

```
!wget https://raw.githubusercontent.com/Sadman-Ishtiak/spam-email-detection-using-using-naive-bayes-classification/refs/heads/
--2026-04-28 19:39:33-- https://raw.githubusercontent.com/Sadman-Ishtiak/spam-email-detection-using-using-naive-bayes-classifi
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8954755 (8.5M) [text/plain]
Saving to: 'emails.csv'

emails.csv      100%[=====>]   8.54M  --.-KB/s    in 0.07s

2026-04-28 19:39:33 (122 MB/s) - 'emails.csv' saved [8954755/8954755]
```

Check the data downloaded successfully

```
!ls

emails.csv  sample_data
```

Dataframe Creation

Then we import pandas and then, convert the csv file into a pandas dataframe into the memory.

```
import pandas as pd
df = pd.read_csv('emails.csv')
df
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1
...	...	...
5723	Subject: re : research and development charges...	0
5724	Subject: re : receipts from visit jim , than...	0
5725	Subject: re : enron case study update wow ! a...	0
5726	Subject: re : interest david , please , call...	0
5727	Subject: news : aurora 5 . 2 update aurora ve...	0

5728 rows x 2 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

Data Balance Check (EDA)

Then we check the balance of the data ie if the numbers of spam and not spam in the dataframe.

```
# print(df.info())
print(f"Number of Spams : {sum(df['spam'] == 1)}")
print(f"Non Spam Number : {sum(df['spam'] == 0)}")
```

```
1368 + 4360 # = 5728 for check
```

```
Number of Spams : 1368
Non Spam Number : 4360
5728
```

✓ Preprocessing the data

We have seen the balance of the data is not proper, but lets process the data by splitting the `text` column into words, wordcount, lowercase, uppcase, symbols, spaces, tabs and line breaks.

```
import string

data = []
for i in range(len(df)):
    row = {}
    row['text'] = df['text'][i]
    row['spam'] = df['spam'][i]
    row['len'] = len(row['text'])
    row['words'] = row['text'].split()
    row['wc'] = len(row['text'].split())
    row['num_digits'] = sum(c.isdigit() for c in row['text'])
    row['num_upper'] = sum(c.isupper() for c in row['text'])
    row['num_lower'] = sum(c.islower() for c in row['text'])
    row['num_punct'] = sum(c in string.punctuation for c in row['text'])
    row['num_spaces'] = sum(c == ' ' for c in row['text'])
    row['num_tabs'] = sum(c == '\t' for c in row['text'])
    row['num_newlines'] = sum(c == '\n' for c in row['text'])
    data.append(row)
df = pd.DataFrame(data)
df.describe()
```

✓ Cleanup zero data

the `num_upper`, `num_tabs` and `num_newlines` dont have any values ie 0. so we can simply delete them.

```
df.drop(['num_upper', 'num_tabs', 'num_newlines'], axis=1, inplace=True)
```

✓ Regression check

Then we check regression for the numeric values with the spam if this is actually works with and we can use a regression or not.

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of DataFrame Features')
plt.show()
```

Which turns out to be a huge fail. as there are no linear relation between spam and any other numeric columns in the dataframe. so we can drop them.

```
df.drop(['num_digits', 'num_lower', 'num_punct', 'num_spaces'], axis=1, inplace=True)
df
```

✓ Word Preprocessor

Since we have found that we cannot use a regression method so the next general advice is to use the words and their relations between the word by getting rid of the same word in different format. (works, work, working, worked ----> work)

```
# Copied from the internet

import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Download necessary NLTK datasets (run these once)
nltk.download('stopwords')
nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))
```

```
def preprocess_spam(text):
    # 2. Clean: Remove non-letters & lowercase
    text = re.sub(r'^a-zA-Z|', ' ', text).lower()

    # 3. Tokenize: Split into words
    words = text.split()

    # 4. Lemmatize & Remove Stopwords
    # Converts 'winning' -> 'win', 'prizes' -> 'prize'
    clean_words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]
    return " ".join(clean_words)

# Example output:
# "Congratulations! You are winning a free prize!"
# -> "congratulation win free prize"
```

✓ Converting the df with the observations that we have had from the previous explorations

```
import string

data = []
for i in range(len(df)):
    row = {}
    row['text'] = preprocess_spam(df['text'][i])
    row['spam'] = df['spam'][i]
    row['len'] = len(row['text'])
    row['words'] = row['text'].split()
    row['wc'] = len(row['text'].split())
    row['num_lower'] = sum(c.islower() for c in row['text'])
    row['num_spaces'] = sum(c == ' ' for c in row['text'])
    row['words'] = row['text'].split()
    data.append(row)
df = pd.DataFrame(data)
df
```

✓ Seeing the data description

```
df.describe()
```

✓ Words and Unique words separation

```
all_words = []

for i in df['words']:
    # print(i)
    for j in i:
        all_words.append(j)
print(len(all_words))
unique_words = set(all_words)
print(len(unique_words))
```

✓ Vectorization of the message text

Since all data science needs to be worked with numerical data we now convert the text of the message into an array of numbers. All the characters are put in a counter with lexicographical order of the unique words.

```
word_to_index = {word: i for i, word in enumerate(sorted(list(unique_words)))}

def custom_vectorize(text_words, vocabulary_map):
    vector = [0] * len(vocabulary_map)
    word_counts = {}

    for word in text_words:
        word_counts[word] = word_counts.get(word, 0) + 1
    for word, count in word_counts.items():
        if word in vocabulary_map:
            vector[vocabulary_map[word]] = count
    return vector

vectorized_data = []
for x in df['words']:
    vectorized_data.append(custom_vectorize(x, word_to_index))
df['vector'] = vectorized_data

df
```

✓ Standard Naive Bayes models

Among the 5 available Naive Bayes models in the scikit learn library we can use the Gaussian, Multinomial, Bernoulli and Complement in this case. So, let's split the data into X, Y axis and convert them into numpy array to see what kinds of accuracy we can get.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB, ComplementNB
from sklearn.metrics import accuracy_score, classification_report

# Fix: Convert the column of lists into a 2D numpy array
X = np.array(df['vector'].tolist())
Y = df['spam']

# train test split
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=2441139)
```

```
print("Gaussian Naive Bayes")
gnb = GaussianNB()
gnb.fit(x_train, y_train)
pred_gnb = gnb.predict(x_test)
print(accuracy_score(y_test, pred_gnb))
print(classification_report(y_test, pred_gnb))
```

```
Gaussian Naive Bayes
0.962478184991274
      precision    recall  f1-score   support

      0       0.97      0.99      0.98       890
      1       0.95      0.88      0.91       256

 accuracy          0.96
macro avg          0.96      0.93      0.94
weighted avg       0.96      0.96      0.96
```

```
print("Multinomial Naive Bayes")
mnb = MultinomialNB()
mnb.fit(x_train, y_train)
pred_mnb = mnb.predict(x_test)
print(accuracy_score(y_test, pred_mnb))
print(classification_report(y_test, pred_mnb))
```

```
Multinomial Naive Bayes
0.9895287958115183
      precision    recall  f1-score   support

      0       1.00      0.99      0.99       890
      1       0.96      0.99      0.98       256

 accuracy          0.99
macro avg          0.98      0.99      0.99
weighted avg       0.99      0.99      0.99
```

```
print("Bernoulli Naive Bayes")
bnb = BernoulliNB()
bnb.fit(x_train, y_train)
pred_bnb = bnb.predict(x_test)
print(accuracy_score(y_test, pred_bnb))
print(classification_report(y_test, pred_bnb))
```

```
Bernoulli Naive Bayes
0.9773123909249564
      precision    recall  f1-score   support

      0       0.98      0.99      0.99       890
      1       0.96      0.94      0.95       256

 accuracy          0.97
macro avg          0.97      0.96      0.97
weighted avg       0.98      0.98      0.98
```

```
print("Complement Naive Bayes")
cnb = ComplementNB()
cnb.fit(x_train, y_train)
pred_cnb = cnb.predict(x_test)
print(accuracy_score(y_test, pred_cnb))
print(classification_report(y_test, pred_cnb))
```

```
Complement Naive Bayes
0.9904013961605584
      precision    recall  f1-score   support

      0       1.00      0.99      0.99       890
      1       0.96      1.00      0.98       256

 accuracy          0.99
macro avg          0.98      0.99      0.99
weighted avg       0.99      0.99      0.99
```

Bagging with the four Models

Although the four model bagging or voting model is not good If it can make a more accurate model it would be better to make and use that. over a single model

```
def bagging(models, x_test, y_test):
    predictions = []
    for model in models:
        pred = model.predict(x_test)
        predictions.append(pred)
    answer = []
    for i in range(len(y_test)):
        temp = []
        for j in range(len(models)):
            temp.append(predictions[j][i])
        answer.append(max(set(temp), key=temp.count))
    return np.array(answer)
```

Testing Precision with the Ensembled voting

```
bagged_pred = bagging([gnb, mnb, bnb, cnb], x_test, y_test)
print(accuracy_score(y_test, bagged_pred))
print(classification_report(y_test, bagged_pred))
```

```
0.9904013961605584
      precision    recall  f1-score   support

     0       0.99      0.99      0.99        890
     1       0.98      0.98      0.98        256

 accuracy          0.99
 macro avg          0.99
weighted avg          0.99
```

Inverting the data with 80% test and 20% learn data

```
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.8, random_state=2441139)
gnbi = GaussianNB()
gnbi.fit(x_train, y_train)
mnbi = MultinomialNB()
mnbi.fit(x_train, y_train)
bnbi = BernoulliNB()
bnbi.fit(x_train, y_train)
cnbi = ComplementNB()
cnbi.fit(x_train, y_train)
bagged_pred = bagging([gnbi, mnbi, bnbi, cnbi], x_test, y_test)
print(accuracy_score(y_test, bagged_pred))
print(classification_report(y_test, bagged_pred))
```

```
0.9522147065241109
      precision    recall  f1-score   support

     0       0.94      1.00      0.97       3496
     1       0.99      0.81      0.89       1087

 accuracy          0.95
 macro avg          0.97
weighted avg          0.95
```